

# Reviewer Pack — Minimal Index of Noncommutative Symmetric Spaces and DPLL Pr...

Entry ee1f6909488a · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-03 09:42:09 UTC

## Minimal Index of Noncommutative Symmetric Spaces and DPLL Proof Width

**Entry ID:** ee1f6909488a **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-03 09:42:09 UTC

### 1. Conjecture

**Field A** (mathematical branch): Noncommutative Geometry (Symmetric Spaces) **Field B** (complexity object): DPLL Search Trees

#### Statement:

For every boolean satisfiability instance  $\varphi$ , the minimal index of the associated noncommutative symmetric space  $X(\varphi)$  is linearly correlated with its DPLL proof width  $w(\varphi)$ , such that  $\text{Index}(X(\varphi)) = \Theta(w(\varphi))$ .

#### Rationale (proposer's reasoning):

Noncommutative symmetries provide a natural framework to study complex systems, and their minimal index could capture the intrinsic complexity of an instance. The DPLL proof width is a well-established measure of complexity in SAT, and this conjecture aims to establish a connection between noncommutative geometry and SAT proof complexity.

**Taxonomy category:** NoncommutativeGeometry (status at proposal time: )

### 2. Pre-registration (Popper-style)

**Hash (SHA-256 prefix):** 52b9706ae6f3d852

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

#### Acceptance criterion:

The conjecture is supported if the correlation coefficient between minimal index of noncommutative symmetric spaces ( $\text{Index}(X(\varphi))$ ) and DPLL proof width ( $w(\varphi)$ ) for all 30 instances is  $\geq 0.8$ , with a p-value  $\leq 0.05$ . The conjecture is falsified if any of these conditions are not met.

### 3. Barrier filter (F1)

**Final:** PASS

| Barrier        | Verdict   | Confidence | LLM <sub>1</sub> | LLM <sub>2</sub> |
|----------------|-----------|------------|------------------|------------------|
| RELATIVIZATION | UNCERTAIN | 1.00       | HITS             | UNCERTAIN        |
| NATURAL_PROOFS | SAFE      | 0.50       | UNCERTAIN        | UNCERTAIN        |
| ALGEBRIZATION  | UNCERTAIN | 1.00       | UNCERTAIN        | HITS             |
| KARP_LIPTON    | SAFE      | 0.50       | UNCERTAIN        | UNCERTAIN        |

## 4. Novelty audit

**Verdict:** NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

## 5. Empirical test harness

**Multi-seed protocol:** 5 seeds (11, 23, 37, 53, 71)

**Sandbox:** isolated subprocess, stdlib-only,  $\leq 90$ s wall time per cycle **Execution:** rc=0, elapsed=0.4s

### 5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def dpll(lits, cls):
        if not lits:
            return True
        lit = lits[0]
        new_lits_true = [l for l in lits if l != -lit and (l < 0 or l not in cls)]
        new_lits_false = [l for l in lits if l != lit and (l > 0 or l not in cls)]
        return dpll(new_lits_true, cls) or dpll(new_lits_false, cls)

    def generate_cnf(n):
        cnf = []
        for _ in range(n):
            clause = random.sample(range(1, n+1), 3)
            cnf.append(clause)
            cnf.append([-lit for lit in clause])
        return cnf

    def solve(cnf, cls):
        return dpll(cnf, cls)

    def min_index(cnf):
        # Placeholder for noncommutative index computation
        return len(cnf)

    def dpll_width(cnf):
        # Placeholder for DPLL proof width computation
        return len(cnf) * 2

    cnf = generate_cnf(10)
    index = min_index(cnf)
    width = dpll_width(cnf)
```



The test code only tests up to  $n = 10$ , which is too small to draw a robust conclusion about the conjecture. The metric 'DPLL Width' is trivially computed as twice the number of clauses, which does not reflect the actual complexity of the problem.

## 9. Final verdict & safety rail

**Verdict:** INCONCLUSIVE

**Reasoning:**

The test code only tests up to  $n = 10$ , which is considered too small by the critic to draw a robust conclusion about the conjecture. Additionally, the metric 'DPLL Width' is trivially computed as twice the number of clauses, not reflecting the actual complexity of the problem. | next: Increase the size of the tested instances and use a more complex metric for DPLL proof width to further investigate the conjecture.

## 11. Audit log (LLM calls)

**Total LLM calls:** 10

| #  | Phase           | Provider      | Model            | Tokens in | out | Latency (ms) |
|----|-----------------|---------------|------------------|-----------|-----|--------------|
| 1  | propose         | ollama_remote | glm4:latest      | 0         | 0   | 17048        |
| 2  | preregistration | ollama_remote | glm4:latest      | 0         | 0   | 28661        |
| 3  | novelty         | ollama_remote | glm4:latest      | 0         | 0   | 11523        |
| 4  | novelty         | ollama_remote | glm4:latest      | 0         | 0   | 14298        |
| 5  | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 15871        |
| 6  | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 13740        |
| 7  | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 10651        |
| 8  | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 123166       |
| 9  | critic          | ollama_remote | glm4:latest      | 0         | 0   | 16275        |
| 10 | judge           | ollama_remote | glm4:latest      | 0         | 0   | 18167        |

**Totals:** 0 input tokens, 0 output tokens, ~\$0.0000 cost, 269400 ms total latency. Provider mix: {'ollama\_remote': 10}

*(full prompt+response transcripts available in research/audit/ee1f6909488a.jsonl)*

## 12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/ee1f6909488a.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

**Sandbox file:** research/pvsnp\_sandbox/test\_\*.py (per-cycle)

**Replay tarball:** research/replay/ee1f6909488a.tar.gz (if generated)